

SWP-PYTHON 5AHWII

WS+SS 2026

Sabo RUBNER - RubnS - sabo.rubner@htlinn.ac.at

15. September 2026

Übersicht

SWP 5A - Python

Test-Termine

Rahmenbedingungen

Stoff-Übersicht

Test-Termine

- ▶ siehe WebUntis
- ▶ Semesterende 2026.12.23 !!!

Github Folien Link

► <https://github.com/htlinnrain/python>

Git Repos G1

► g1

Git Repos G2

▶ g2

Rahmenbedingungen

- ▶ drei oder vier Tests
- ▶ mehr als die Hälfte der Einheiten gefehlt - P5 Prüfung
- ▶ zu spät kommen - Tafel
- ▶ Mitarbeit: jeder startet bei Befriedigend, plus -0.25 und minus +0.5
- ▶ Mitarbeit 30% Test 70% Gesamtnote - 1HJ 45% 2HJ 55%
- ▶ jeder Test muss positiv sein, wenn nicht P5 Prüfung
- ▶ Teststoff siehe Fragensammlung
- ▶ HUs gehören in ein eigenes GIT Repo

Stoff-Übersicht

- ▶ Python Theorie und einzelne Aufgaben
- ▶ verkettete Listen mit Python
- ▶ 2tes HJ individuelles Projekt in Python

Ablauf KW38

- ▶ Einführung
- ▶ Folien via GIT
- ▶ Git Repo für 2026 SWP-PYTHON anlegen
- ▶ Python2/Python3 (in P3 mit Abwärtskompatibilität gebrochen)
- ▶ Python downloaden, HalloWelt programmieren
- ▶ iterativ und rekursiv besprechen
- ▶ Unterschied Prozesse Threads

Ablauf KW38

- ▶ **Wer den Code nicht versteht, steuert nicht – er fliegt blind**
 - ▶ **Debugging & Verifikation:** KI generiert schnell funktionierenden Code, baut aber regelmäßig subtile Logikfehler, Race Conditions oder Sicherheitslücken ein. Wer Grundlagen, Datenstrukturen und Systemverhalten nicht beherrscht, kann KI-Code weder prüfen noch reparieren.
 - ▶ **Die Pilot-Analogie:** Ein moderner Jet fliegt meist auf Autopilot. Trotzdem muss der Pilot das Flugzeug im Detail verstehen, um bei Systemausfällen oder Turbulenzen die Kontrolle zu übernehmen.
- ▶ **Programmieren ist Problemübersetzung, nicht Syntax-Tippen**
 - ▶ **Architektur & Kontext:** Das Generieren von Syntax macht nur einen Bruchteil der Softwareentwicklung aus. Die eigentliche Ingenieursleistung liegt im Systementwurf, dem Schnittstellendesign, der Leistungsskalierung und dem Verstehen der realen Problemstellung.
 - ▶ **Präzise Anforderung:** Um einer KI überhaupt den richtigen Auftrag (Prompt) zu geben, muss das Problem auf technischer Ebene bereits durchdrungen sein. Vage Eingaben führen zu unbrauchbarem Code.

Ablauf KW38

► Haftung und der Multiplikator-Effekt

- **Verantwortung:** Bei Fehlern in kritischen Systemen (Medizintechnik, Steuerungs- und Regelungstechnik, Automobil oder Finanzen) haftet kein Sprachmodell, sondern der Entwickler mit seiner Unterschrift.
- **Neue Rollenverteilung:** KI eliminiert rein mechanische Tipparbeit („Code Monkeys“). Entwickler, die Informatikgrundlagen beherrschen und KI als Werkzeug einsetzen, werden nicht arbeitslos, sondern um ein Vielfaches produktiver.